МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное автономное образовательное учреждение высшего образования

Санкт-Петербургский национальный исследовательский университет ИТМО

Мегафакультет трансляционных информационных технологий

Факультет информационных технологий и программирования

Отчет по лабораторной работе №5

По дисциплине “Администрирование Linux”

Выполнил студент группы M3303

Лукин Даниил Валерьевич

Проверил

Гумбатов Владислав Юрьевич

УНИВЕРСИТЕТ ИТМО

Санкт-Петербург

2026

Задание

Создайте текстовый файл, в котором запишете последовательность команд для выполнения каждого из нижеследующих заданий. Для команд, имеющих интерактивный интерфейс, опишите последовательность выбора управляющих команд и их параметров. Если решение заключается в изменении конфигурационного файла, укажите название файла и вносимые правки. Во многих заданиях будет фигурировать **ID** – это последние 2 цифры вашего номера студента в ИСУ.

1. Квоты на процессор для конкретного пользователя (cgroups v2)

- Создайте пользователя: `user-ID` (например, `user-72`).
- Назначьте квоту процессора на основе номера пользователя:
  - Если имя пользователя заканчивается на **0-30**: 30% CPU.
  - Если имя пользователя заканчивается на **31-70**: 50% CPU.
  - Если имя пользователя заканчивается на **71-99**: 70% CPU.

2. Ограничение памяти для процесса (cgroups)

- Создайте cgroup для ограничения памяти, потребляемой процессом.
- Запустите процесс и переместите его в созданную вами группу.
  - Пример команды: `tail /dev/zero`.
- Ограничьте потребление памяти следующим образом: `ID*10 + 500 МБ` (например, ID=23 → 730 МБ).
- Проверьте, что при исчерпании памяти процессом он прерывается ОС.

3. Ограничение дискового ввода-вывода для сценария резервного копирования (cgroups)

- Скрипт резервного копирования (`backup.sh`) перегружает дисковую подсистему.
- Ограничьте его до:
  - Чтение: `1000 + <ID>*10 IOPS`.
  - Запись: `500 + <ID>*10 IOPS`.
- Используйте `cgcreate` для установки ограничений `io.max`.
- Протестируйте с помощью `fio` или `dd`.

4. Закрепление к определенному ядру процессора для приложения

- Настройте с помощью cgroups процесс команды `top` за процессором 0.
- Используйте `cpuset.cpus` в cgroups.
- Проверьте с помощью `taskset -p <PID>`. (требуется пакет `sysstat`)

5. Динамическая корректировка ресурсов (cgroups)

- Напишите сценарий для мониторинга нагрузки по CPU и динамического изменения `cpu.max` определенного процесса (его идентификатор задается как входной параметр скрипта).
- Квота ЦП для процесса должна регулироваться в зависимости от общей нагрузки на систему:
  - Низкая нагрузка (CPU < 20%): 80% CPU.
  - Высокая нагрузка (CPU > 60%): 30% CPU.

6. Создание изолированного имени хоста (пространство имен UTS)

- Запустите оболочку (`shell/bash`) в отдельном namespace, в которой можно изменить имя хоста, не затрагивая хост.
- Измените имя хоста внутри пространства имен на `isolated-student-<ID>`.
- Проверьте изоляцию:

- `hostname` - Должно отображаться «isolated-student-».
- Проверьте имя хоста (в новом терминале): `hostname` - По-прежнему показывает оригинальное имя хоста.

7. Изоляция процессов (пространство имен PID)

- Создайте пространство имен, в котором процессы хоста будут невидимы:
  - `unshare --pid --fork bash`.
- Смонтируйте новый каталог `/proc`:
  - `mount -t proc proc /proc`.
- Проверьте процессы:
  - `ps aux` = Показывает только 2-3 процесса (например, `bash`, `ps`).
- Сравните с хостом (в новом терминале):
  - `ps aux` = Показывает все процессы хоста.

8. Изолированная файловая система (пространство имен Mount)

- Создайте каталог, видимый только в пространстве имен:
  - `unshare --mount bash`.
- Создайте приватный каталог:
  - `mkdir /tmp/private_$(whoami)`.
- Смонтируйте временную файловую систему:
  - `mount -t tmpfs tmpfs /tmp/private_$(whoami)`.
- Проверьте изоляцию:
  - `df -h | grep private_$(whoami)` = Запишите в отчет результат.
- Проверка на хосте (в новом терминале):
  - `df -h | grep private_$(whoami)`.

9. Отключение доступа к сети (пространство имен Network)

- Запустите командный интерпретатор `bash` без доступа к сети.
- Проверьте сетевые интерфейсы:
  - `ip addr` = Запишите в отчет, что показывает команда.
- Проверьте подключение:
  - `ping google.com`.
- Сравните с хостом (в новом терминале):
  - `ping google.com`.

10. Создайте и проанализируйте монтирование OverlayFS

Шаги:

а. Первоначальная настройка:

- Создайте каталоги:

```
mkdir -p ~/overlay_<ID>/{lower,upper,work,merged}
```

- В каталоге `lower` создайте файл с именем `<ID>_original.txt` с содержанием:

```
Оригинальный текст из LOWER
```

- Смонтируйте OverlayFS:

```
mount -t overlay overlay -o lowerdir=~/overlay_<ID>/lower,upperdir=~/overlay_<ID>/
upper,workdir=~/overlay_<ID>/work ~/overlay_<ID>/merged
```

b. Имитация неполадки и отладка:

- Удалите файл `<ID>_original.txt` из каталога `merged`.
- Понаблюдайте: Какой файл(ы) появился(ись) в верхнем каталоге? Задокументируйте их имена и содержимое.
- Измените каталог `merged`, чтобы **восстановить** `<ID>_original.txt`, не размонтируя и не изменяя нижний уровень.

c. Разработайте скрипт, который:

- Обнаруживает все `whiteout` файлы в верхнем каталоге `upper`.
- Сравнивает содержимое нижнего и объединенного для выявления несоответствий.
- Выводит отчет с именем `<ID>_audit.log`.

d. Ответьте на вопросы:

- Как OverlayFS скрывает файлы из нижнего слоя при удалении в объединенном?
- Если вы удалите рабочий каталог `work`, сможете ли вы перемонтировать оверлей? Объясните, почему.
- Что произойдет с объединенным слоем, если верхний каталог будет пуст?

11. Оптимизируйте Dockerfile для приведенного ниже приложения app.py

```python
from flask import Flask
import socket
import os

app = Flask(__name__)

@app.route('/')
def container_info():
    # Get container IP
    hostname = socket.gethostname()
    ip_address = socket.gethostbyname(hostname)
    # Get student name from environment variable
    student_name = os.getenv('STUDENT_NAME', 'Rincewind')
    return f"Container IP: {ip_address} Student: {student_name}"

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
```

Исходный Dockerfile:

```dockerfile
FROM python:latest
COPY . /app
WORKDIR /app
RUN pip install flask
CMD ["python", "app.py"]
```

Улучшите Dockerfile с учетом лучших практик:

- Используйте меньший базовый образ.

- Зафиксируйте версию образа.
- Запуск от имени пользователя, не являющегося root.
- Используйте кэширование слоев для зависимостей.
- Добавьте файл `.dockerignore`.

—

12. Установка платформы публикации WordPress с помощью Docker Compose

Задача: Создать `docker-compose.yml` для запуска WordPress и MySQL/MariaDB с сохранением состояния при перезапуске контейнеров.

Используйте:

- Порт `<ID>+2000` для WordPress (например, ID = 65 ⟶ port = 2065).
- Пароль базы данных: `[ваше_имя]_db_pass`.
- Том с именем `[ваше_имя]-wp-data` для WordPress.

Выполнение работы

Часть 1. Квоты на процессор для конкретного пользователя (cgroups v2)

- Создан пользователь user-31: sudo useradd -m user-31
- Создана cgroup: mkdir /sys/fs/cgroup/user-31
- Установлено ограничение CPU: echo “50000 100000” > /sys/fs/cgroup/user-31/cpu.max
- Назначена квота CPU через systemd: systemctl set-property user-31.slice CPUQuota=50%

Проверка выполнена с помощью утилиты stress. Нагрузка процесса не превышает 50% CPU.

```
[03-04-2026 08:32:20] {44fcaf38} student@vm-amd:~$ sudo useradd -m user-31
[sudo] password for student:
[03-04-2026 08:32:36] {44fcaf38} student@vm-amd:~$ id user-31
uid=1001(user-31) gid=1001(user-31) groups=1001(user-31)
[03-04-2026 08:32:41] {44fcaf38} student@vm-amd:~$ mount | grep cgroup
cgroup2 on /sys/fs/cgroup type cgroup2 (rw,nosuid,nodev,noexec,relatime,nsdelegate,memory_recursiveprot)
[03-04-2026 08:32:48] {44fcaf38} student@vm-amd:~$ sudo mkdir -p /sys/fs/cgroup/user-31
[03-04-2026 08:32:57] {44fcaf38} student@vm-amd:~$ echo "50000 100000" | sudo tee /sys/fs/cgroup/user-31/cpu.max
50000 100000
[03-04-2026 08:33:03] {44fcaf38} student@vm-amd:~$ sudo systemctl set-property user-31.slice CPUQuota=50%
[03-04-2026 08:33:09] {44fcaf38} student@vm-amd:~$ systemctl show user-31.slice | grep CPUQuota
CPUQuotaPerSecUSec=500ms
CPUQuotaPeriodUSec=infinity
DropInPaths=/usr/lib/systemd/system/user-.slice.d/10-defaults.conf /etc/systemd/system.control/user-31.slice.d/50-CPUQuota.conf
```

```
[03-04-2026 08:33:25] {44fcaf38} student@vm-amd:~$ sudo -u user-31 stress --cpu 2
stress: info: [3783] dispatching hogs: 2 cpu, 0 io, 0 vm, 0 hdd
```

```
top - 08:36:31 up 8 min,  5 users,  load average: 0.79, 0.24, 0.09
Tasks: 114 total,   3 running, 111 sleeping,   0 stopped,   0 zombie
%Cpu(s):100.0 us,  0.0 sy,  0.0 ni,  0.0 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
MiB Mem :   1968.1 total,    525.2 free,    383.5 used,   1223.5 buff/cache
MiB Swap:      0.0 total,      0.0 free,      0.0 used.   1584.6 avail Mem

    PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TIME+ COMMAND
   3785 user-31   20   0    3620    568    460 R  50.3   0.0   0:15.68 stress
   3784 user-31   20   0    3620    568    460 R  49.7   0.0   0:15.67 stress
     27 root      20   0       0      0      0 I   0.3   0.0   0:00.11 kworker/u2:2-events_freezable_+
```

Часть 2. Ограничение памяти для процесса (cgroups v2)

- Создана cgroup: mkdir /sys/fs/cgroup/memory_limit_31
- Установлен лимит памяти: echo 849346560 > memory.max
- Запущен процесс: tail /dev/zero &
- Процесс добавлен в cgroup: echo > cgroup.procs
- Для тестирования использована утилита stress: stress –vm 1 –vm-bytes 900M

При превышении лимита процесс завершён системой (OOM killer).

```
[03-04-2026 08:43:37] {44fcaf38} student@vm-amd:~$ sudo mkdir -p /sys/fs/cgroup/memory_limit_31
[03-04-2026 08:43:43] {44fcaf38} student@vm-amd:~$ echo 849346560 | sudo tee /sys/fs/cgroup/memory_limit_31/memory.max
849346560
[03-04-2026 08:43:51] {44fcaf38} student@vm-amd:~$ cat /sys/fs/cgroup/memory_limit_31/memory.max
849346560
```

```
[03-04-2026 08:41:24] {44fcaf38} student@vm-amd:~$ stress --vm 1 --vm-bytes 900M
stress: info: [3966] dispatching hogs: 0 cpu, 0 io, 1 vm, 0 hdd
stress: FAIL: [3966] (425) <-- worker 3967 got signal 9
stress: WARN: [3966] (427) now reaping child worker processes
stress: FAIL: [3966] (451) failed run completed in 29s
[03-04-2026 08:42:09] {44fcaf38} student@vm-amd:~$
```

```
[03-04-2026 08:41:46] {44fcaf38} student@vm-amd:~$ echo 3967 | sudo tee /sys/fs/cgroup/memory_limit_31/
cgroup.procs
[sudo] password for student:
3967
[03-04-2026 08:42:09] {44fcaf38} student@vm-amd:~$
```

```
[03-04-2026 08:41:24] {44fcaf38} student@vm-amd:~$ stress --vm 1 --vm-bytes 900M
stress: info: [3966] dispatching hogs: 0 cpu, 0 io, 1 vm, 0 hdd
stress: FAIL: [3966] (425) <-- worker 3967 got signal 9
stress: WARN: [3966] (427) now reaping child worker processes
stress: FAIL: [3966] (451) failed run completed in 29s
[03-04-2026 08:42:09] {44fcaf38} student@vm-amd:~$ stress --vm 1 --vm-bytes 800M
stress: info: [3973] dispatching hogs: 0 cpu, 0 io, 1 vm, 0 hdd
```

```
[03-04-2026 08:42:28] {44fcaf38} student@vm-amd:~$ echo 3974 | sudo tee /sys/fs/cgroup/memory_limit_31/
cgroup.procs
3974
[03-04-2026 08:42:36] {44fcaf38} student@vm-amd:~$
```

Часть 3. Ограничение дискового ввода-вывода для сценария резервного копирования (cgroups)

- Определено устройство диска: lsblk ⟶ 8:0
- Создана cgroup: mkdir /sys/fs/cgroup/io_limit_31
- Установлены ограничения: echo “8:0 riops=1310 wiops=810” > io.max
- Проверка выполнена с помощью fio.

```
#!/bin/bash
sudo cgexec -g io:io_limit_31 fio --name=write_test --filename=testfile --size=1G --bs=4k --rw=write --direct=1 --iodepth=1 --runtime=10 --time_based
sudo cgexec -g io:io_limit_31 fio --name=read_test --filename=testfile --size=1G --bs=4k --rw=read --direct=1 --iodepth=1 --runtime=10 --time_based
```

```
[03-04-2026 08:44:07] {44fcaf38} student@vm-amd:~$ lsblk
NAME   MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
sda      8:0    0   15G  0 disk
├─sda1   8:1    0    1M  0 part
└─sda2   8:2    0   15G  0 part /
sr0     11:0    1 1024M  0 rom
[03-04-2026 08:45:19] {44fcaf38} student@vm-amd:~$ sudo mkdir -p /sys/fs/cgroup/io_limit_31
[03-04-2026 08:45:27] {44fcaf38} student@vm-amd:~$ echo "8:0 riops=1310 wiops=810" | sudo tee /sys/fs/cgroup/io_limit_31/io.max
8:0 riops=1310 wiops=810
```

```
[03-04-2026 09:05:53] {44fcaf38} student@vm-amd:~$ vim backup.sh
[03-04-2026 09:06:17] {44fcaf38} student@vm-amd:~$ cat backup.sh
#!/bin/bash
sudo cgexec -g io:io_limit_31 fio --name=write_test --filename=testfile --size=1G --bs=4k --rw=write --direct=1 --iodepth=1 --runtime=10 --time_based
sudo cgexec -g io:io_limit_31 fio --name=read_test --filename=testfile --size=1G --bs=4k --rw=read --direct=1 --iodepth=1 --runtime=10 --time_based
```

```
[03-04-2026 09:05:31] {44fcaf38} student@vm-amd:~$ ./backup.sh
write_test: (g=0): rw=write, bs=(R) 4096B-4096B, (W) 4096B-4096B, (T) 4096B-4096B, ioengine=psync, iodepth=1
fio-3.36
Starting 1 process
Jobs: 1 (f=1): [W(1)][100.0%][w=3243KiB/s][w=810 IOPS][eta 00m:00s]
write_test: (groupid=0, jobs=1): err= 0: pid=31351: Fri Apr  3 09:05:43 2026
  write: IOPS=801, BW=3205KiB/s (3282kB/s)(31.3MiB/10009msec); 0 zone resets
    clat (usec): min=28, max=101567, avg=1230.47, stdev=9190.58
     lat (usec): min=28, max=101567, avg=1231.08, stdev=9190.57
    clat percentiles (usec):
     |  1.00th=[    30],  5.00th=[    31], 10.00th=[    33], 20.00th=[    42],
     | 30.00th=[   172], 40.00th=[   217], 50.00th=[   239], 60.00th=[   265],
     | 70.00th=[   285], 80.00th=[   293], 90.00th=[   318], 95.00th=[   363],
     | 99.00th=[ 73925], 99.50th=[ 83362], 99.90th=[ 98042], 99.95th=[100140],
     | 99.99th=[101188]
   bw (  KiB/s): min= 2592, max= 3856, per=100.00%, avg=3220.26, stdev=293.75, samples=19
   iops        : min=  648, max=  964, avg=805.05, stdev=73.44, samples=19
  lat (usec)   : 50=22.36%, 100=5.44%, 250=27.96%, 500=42.44%, 750=0.39%
  lat (usec)   : 1000=0.02%
  lat (msec)   : 2=0.11%, 4=0.02%, 10=0.02%, 100=1.16%, 250=0.07%
  cpu          : usr=0.75%, sys=8.10%, ctx=8025, majf=0, minf=12
  IO depths    : 1=100.0%, 2=0.0%, 4=0.0%, 8=0.0%, 16=0.0%, 32=0.0%, >=64=0.0%
     submit    : 0=0.0%, 4=100.0%, 8=0.0%, 16=0.0%, 32=0.0%, 64=0.0%, >=64=0.0%
     complete  : 0=0.0%, 4=100.0%, 8=0.0%, 16=0.0%, 32=0.0%, 64=0.0%, >=64=0.0%
     issued rwts: total=0,8020,0,0 short=0,0,0,0 dropped=0,0,0,0
     latency   : target=0, window=0, percentile=100.00%, depth=1

Run status group 0 (all jobs):
  WRITE: bw=3205KiB/s (3282kB/s), 3205KiB/s-3205KiB/s (3282kB/s-3282kB/s), io=31.3MiB (32.8MB), run=10009-10009msec

Disk stats (read/write):
  sda: ios=0/7942, sectors=0/63632, merge=0/12, ticks=0/1204, in_queue=1226, util=11.33%
read_test: (g=0): rw=read, bs=(R) 4096B-4096B, (W) 4096B-4096B, (T) 4096B-4096B, ioengine=psync, iodepth=1
fio-3.36
Starting 1 process
Jobs: 1 (f=1): [R(1)][100.0%][r=5245KiB/s][r=1311 IOPS][eta 00m:00s]
read_test: (groupid=0, jobs=1): err= 0: pid=31357: Fri Apr  3 09:05:53 2026
  read: IOPS=1309, BW=5238KiB/s (5363kB/s)(51.2MiB/10005msec)
    clat (usec): min=27, max=95892, avg=759.44, stdev=5066.44
     lat (usec): min=27, max=95892, avg=759.85, stdev=5066.44
    clat percentiles (usec):
     |  1.00th=[    67],  5.00th=[    72], 10.00th=[    74], 20.00th=[    82],
     | 30.00th=[   101], 40.00th=[   289], 50.00th=[   433], 60.00th=[   474],
     | 70.00th=[   515], 80.00th=[   553], 90.00th=[   603], 95.00th=[   627],
```

Часть 4. Закрепление к определенному ядру процессора для приложения

- Создана cgroup: mkdir /sys/fs/cgroup/cpuset_top_31
- Настроены параметры:
  - echo 0 > cpuset.cpus
  - echo 0 > cpuset.mems

```
[03-04-2026 09:13:26] {44fcaf38} student@vm-amd:~$ sudo mkdir -p /sys/fs/cgroup/cpuset_top_31
[03-04-2026 09:13:31] {44fcaf38} student@vm-amd:~$ nproc
1
[03-04-2026 09:13:34] {44fcaf38} student@vm-amd:~$ echo 0 | sudo tee /sys/fs/cgroup/cpuset_top_31/cpuset.mems
0
```

```
[03-04-2026 09:11:39] {44fcaf38} student@vm-amd:~$ stress --vm 1 --vm-bytes 800M
stress: info: [31465] dispatching hogs: 0 cpu, 0 io, 1 vm, 0 hdd
i
```

```
[03-04-2026 09:11:30] {44fcaf38} student@vm-amd:~$ top -n 1 | grep stress
  31466 student   20   0  822824 724452    460 R  90.9  35.9   0:09.34 stress
[03-04-2026 09:12:03] {44fcaf38} student@vm-amd:~$ echo 31466 | sudo tee /sys/fs/cgroup/cpuset_top_31/c
group.procs
[sudo] password for student:
31466
[03-04-2026 09:12:20] {44fcaf38} student@vm-amd:~$ taskset -p 31466
pid 31466's current affinity mask: 1
[03-04-2026 09:12:33] {44fcaf38} student@vm-amd:~$
```

Часть 5. Динамическая корректировка ресурсов (cgroups)

- Создана cgroup: mkdir /sys/fs/cgroup/dynamic_cpu_31
- Разработан скрипт cpu_control.sh для мониторинга нагрузки CPU и изменения cpu.max.
- Логика:
  - CPU < 20% ⟶ 80% CPU
  - CPU > 60% ⟶ 30% CPU

```
[03-04-2026 09:13:38] {44fcaf38} student@vm-amd:~$ sudo mkdir -p /sys/fs/cgroup/dynamic_cpu_31
[03-04-2026 09:15:56] {44fcaf38} student@vm-amd:~$ vim cpu_control.sh
```

```
top - 09:20:34 up 52 min,  5 users,  load average: 0.91, 0.54, 0.40
Tasks: 116 total,   2 running, 114 sleeping,   0 stopped,   0 zombie
%Cpu(s):  5.3 us, 14.7 sy,  0.0 ni, 80.0 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
MiB Mem :   1968.1 total,   1230.1 free,    625.2 used,    293.2 buff/cache
MiB Swap:      0.0 total,      0.0 free,      0.0 used.   1342.8 avail Mem

    PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TIME+ COMMAND
  31663 student   20   0  822824 100868    460 R  79.4   5.0   1:01.27 stress
      1 root      20   0   22608  12600   8332 S   0.0   0.6   0:04.13 systemd
      2 root      20   0       0      0      0 S   0.0   0.0   0:00.00 kthreadd
      3 root      20   0       0      0      0 S   0.0   0.0   0:00.00 pool_workqueue_release
      4 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-rcu_g
      5 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-rcu_p
      6 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-slub_
      7 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-netns
      9 root      20   0       0      0      0 I   0.0   0.0   0:00.87 kworker/0:1-events
     12 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-mm_pe
     13 root      20   0       0      0      0 I   0.0   0.0   0:00.00 rcu_tasks_kthread
     14 root      20   0       0      0      0 I   0.0   0.0   0:00.00 rcu_tasks_rude_kthread
     15 root      20   0       0      0      0 I   0.0   0.0   0:00.00 rcu_tasks_trace_kthread
     16 root      20   0       0      0      0 S   0.0   0.0   0:00.75 ksoftirqd/0
     17 root      20   0       0      0      0 I   0.0   0.0   0:00.62 rcu_preempt
     18 root      rt   0       0      0      0 S   0.0   0.0   0:00.02 migration/0
     19 root     -51   0       0      0      0 S   0.0   0.0   0:00.00 idle_inject/0
     20 root      20   0       0      0      0 S   0.0   0.0   0:00.00 cpuhp/0
```

```
[03-04-2026 09:18:58] {44fcaf38} student@vm-amd:~$ stress --vm 1 --vm-bytes 800M
stress: info: [31662] dispatching hogs: 0 cpu, 0 io, 1 vm, 0 hdd
```

```
[03-04-2026 09:19:09] {44fcaf38} student@vm-amd:~$ top -n 1 | grep stress
[03-04-2026 09:19:12] {44fcaf38} student@vm-amd:~$ top -n 1 | grep stress
  31663 student   20   0  822824 691624    460 R  99.9  34.3   0:02.61 stress
[03-04-2026 09:19:20] {44fcaf38} student@vm-amd:~$ sudo ./cpu_control.sh 31663
[sudo] password for student:
Monitoring CPU load...
CPU load: 100%
Setting CPU limit to 30%
CPU load: 2%
Setting CPU limit to 80%
CPU load: 19%
Setting CPU limit to 80%
CPU load: 29%
CPU load: 31%
CPU load: 28%
CPU load: 23%
CPU load: 20%
CPU load: 70%
Setting CPU limit to 30%
CPU load: 5%
Setting CPU limit to 80%
CPU load: 28%
CPU load: 23%
CPU load: 24%
CPU load: 40%
CPU load: 23%
CPU load: 22%
CPU load: 20%
CPU load: 20%
CPU load: 20%
CPU load: 20%
```

```bash
#!/bin/bash
CGROUP="/sys/fs/cgroup/dynamic_cpu_31"
PID=$1
if [ -z "$PID" ]; then
    echo "Usage: $0 <PID>"
    exit 1
fi

echo $PID | sudo tee $CGROUP/cgroup.procs > /dev/null
echo "Monitoring CPU load..."

while true; do
    CPU_LOAD=$(mpstat 1 1 | awk '/Average/ {print 100 - $NF}')
    CPU_LOAD_INT=${CPU_LOAD%.*}
    echo "CPU load: $CPU_LOAD_INT%"
    if [ "$CPU_LOAD_INT" -lt 20 ]; then
        echo "Setting CPU limit to 80%"
        echo "80000 100000" | sudo tee $CGROUP/cpu.max > /dev/null
    elif [ "$CPU_LOAD_INT" -gt 60 ]; then
        echo "Setting CPU limit to 30%"
        echo "30000 100000" | sudo tee $CGROUP/cpu.max > /dev/null
    fi
    sleep 2
done
```

Часть 6. Создание изолированного имени хоста (пространство имен UTS)

- Создан namespace: unshare –uts –fork bash
- Изменено имя хоста: hostname isolated-student-31
- Проверка: hostname ⟶ isolated-student-31
- На хосте: hostname ⟶ исходное имя

```
[03-04-2026 09:22:49] {44fcaf38} student@vm-amd:~$ sudo unshare --uts --fork bash
[03-04-2026 09:22:52] {44fcaf38} root@vm-amd:/home/student# hostname isolated-student-31
[03-04-2026 09:22:59] {44fcaf38} root@vm-amd:/home/student# hostname
isolated-student-31
[03-04-2026 09:23:07] {44fcaf38} root@vm-amd:/home/student#
```

```
[03-04-2026 09:23:21] {44fcaf38} student@vm-amd:~$ hostname
vm-amd
[03-04-2026 09:23:24] {44fcaf38} student@vm-amd:~$
```

Часть 7. Изоляция процессов (пространство имен PID)

- Создан namespace: unshare –pid –fork bash
- Смонтирован /proc: mount -t proc proc /proc
- Проверка: ps aux ⟶ отображается ограниченное число процессов
- На хосте: ps aux ⟶ отображаются все процессы

```
[03-04-2026 09:31:36] {44fcaf38} student@vm-amd:~$ sudo unshare --pid --fork --mount bash
[03-04-2026 09:33:03] {44fcaf38} root@vm-amd:/home/student# mount -t proc proc /proc
[03-04-2026 09:33:10] {44fcaf38} root@vm-amd:/home/student# ps aux
USER         PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root           1  0.0  0.2   7604  4444 pts/5    S    09:33   0:00 bash
root          11  0.0  0.2  10884  4568 pts/5    R+   09:33   0:00 ps aux
[03-04-2026 09:33:14] {44fcaf38} root@vm-amd:/home/student#
```

```
[03-04-2026 09:33:19] {44fcaf38} student@vm-amd:~$ ps aux
USER         PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root           1  0.1  0.6  22608 12900 ?        Ss   08:32   0:04 /usr/lib/systemd/systemd --system --
root           2  0.0  0.0      0     0 ?        S    08:32   0:00 [kthreadd]
root           3  0.0  0.0      0     0 ?        S    08:32   0:00 [pool_workqueue_release]
root           4  0.0  0.0      0     0 ?        I<   08:32   0:00 [kworker/R-rcu_g]
root           5  0.0  0.0      0     0 ?        I<   08:32   0:00 [kworker/R-rcu_p]
root           6  0.0  0.0      0     0 ?        I<   08:32   0:00 [kworker/R-slub_]
root           7  0.0  0.0      0     0 ?        I<   08:32   0:00 [kworker/R-netns]
root           9  0.0  0.0      0     0 ?        I    08:32   0:01 [kworker/0:1-events]
root          12  0.0  0.0      0     0 ?        I<   08:32   0:00 [kworker/R-mm_pe]
root          13  0.0  0.0      0     0 ?        I    08:32   0:00 [rcu_tasks_kthread]
root          14  0.0  0.0      0     0 ?        I    08:32   0:00 [rcu_tasks_rude_kthread]
root          15  0.0  0.0      0     0 ?        I    08:32   0:00 [rcu_tasks_trace_kthread]
root          16  0.0  0.0      0     0 ?        S    08:32   0:00 [ksoftirqd/0]
root          17  0.0  0.0      0     0 ?        R    08:32   0:00 [rcu_preempt]
root          18  0.0  0.0      0     0 ?        S    08:32   0:00 [migration/0]
root          19  0.0  0.0      0     0 ?        S    08:32   0:00 [idle_inject/0]
root          20  0.0  0.0      0     0 ?        S    08:32   0:00 [cpuhp/0]
root          21  0.0  0.0      0     0 ?        S    08:32   0:00 [kdevtmpfs]
```

Часть 8. Изолированная файловая система (пространство имен Mount)

- Создан namespace: unshare –mount bash
- Настроен приватный режим: mount –make-rprivate /
- Создан каталог: `/tmp/private_$(whoami)`
- Смонтирована : `mount -t tmpfs tmpfs /tmp/private_$(whoami)`
- Проверка внутри: df -h ⟶ tmpfs отображается
- Проверка снаружи: df -h ⟶ отсутствует

```
[03-04-2026 09:34:43] {44fcaf38} student@vm-amd:~$ sudo unshare --mount bash
[03-04-2026 09:34:47] {44fcaf38} root@vm-amd:/home/student# mkdir /tmp/private_$(whoami)
[03-04-2026 09:35:09] {44fcaf38} root@vm-amd:/home/student# mount -t tmpfs tmpfs /tmp/private_$(whoami)
[03-04-2026 09:35:18] {44fcaf38} root@vm-amd:/home/student# df -h | grep private_$(whoami)
tmpfs           985M     0  985M   0% /tmp/private_root
[03-04-2026 09:35:25] {44fcaf38} root@vm-amd:/home/student#
```

```
[03-04-2026 09:35:55] {44fcaf38} student@vm-amd:~$ df -h | grep private_$(whoami)
[03-04-2026 09:35:58] {44fcaf38} student@vm-amd:~$
```

Часть 9. Отключение доступа к сети (пространство имен Network)

- Создан namespace: unshare –net bash
- Проверка интерфейсов: ip addr ⟶ доступен только lo
- Проверка сети: ping google.com ⟶ сеть недоступна
- На хосте: ping работает

```
[03-04-2026 09:37:18] {44fcaf38} student@vm-amd:~$ sudo unshare --net bash
[03-04-2026 09:37:20] {44fcaf38} root@vm-amd:/home/student# ip addr
1: lo: <LOOPBACK> mtu 65536 qdisc noop state DOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
[03-04-2026 09:37:27] {44fcaf38} root@vm-amd:/home/student# ping google.com
ping: google.com: Temporary failure in name resolution
[03-04-2026 09:37:56] {44fcaf38} root@vm-amd:/home/student#
```

```
[03-04-2026 09:38:00] {44fcaf38} student@vm-amd:~$ ping google.com
PING google.com (74.125.131.138) 56(84) bytes of data.
64 bytes from lu-in-f138.1e100.net (74.125.131.138): icmp_seq=1 ttl=255 time=8.41 ms
64 bytes from lu-in-f138.1e100.net (74.125.131.138): icmp_seq=2 ttl=255 time=6.68 ms
64 bytes from lu-in-f138.1e100.net (74.125.131.138): icmp_seq=3 ttl=255 time=6.88 ms
64 bytes from lu-in-f138.1e100.net (74.125.131.138): icmp_seq=4 ttl=255 time=7.94 ms
64 bytes from lu-in-f138.1e100.net (74.125.131.138): icmp_seq=5 ttl=255 time=7.80 ms
64 bytes from lu-in-f138.1e100.net (74.125.131.138): icmp_seq=6 ttl=255 time=7.68 ms
^C
--- google.com ping statistics ---
6 packets transmitted, 6 received, 0% packet loss, time 5009ms
rtt min/avg/max/mdev = 6.682/7.554/8.410/0.601 ms
[03-04-2026 09:38:08] {44fcaf38} student@vm-amd:~$
```

Часть 10. Создайте и проанализируйте монтирование OverlayFS

- Создан namespace: unshare –mount bash
- Настроен приватный режим: mount –make-rprivate /
- Создан каталог: `/tmp/private_$(whoami)`
- Смонтирована : `mount -t tmpfs tmpfs /tmp/private_$(whoami)`
- Проверка внутри: df -h ⟶ tmpfs отображается
- Проверка снаружи: df -h ⟶ отсутствует

```
[03-04-2026 09:51:15] {44fcaf38} student@vm-amd:~$ mkdir -p ~/overlay_31/{lower,upper,work,merged}
[03-04-2026 09:51:30] {44fcaf38} student@vm-amd:~$ echo "Сригинальный текст из LOWER" > ~/overlay_31/lower/31_original.txt
[03-04-2026 09:51:34] {44fcaf38} student@vm-amd:~$ sudo mount -t overlay overlay -o lowerdir=/home/student/overlay_31/lower,upperdir=/home/student/overlay_31/upper,workdir=/home/student/overlay_31/work /home/
student/overlay_31/merged
[03-04-2026 09:51:47] {44fcaf38} student@vm-amd:~$ ls /home/student/overlay_31/merged
31_original.txt
[03-04-2026 09:51:55] {44fcaf38} student@vm-amd:~$ df -h ~/overlay_31
Filesystem      Size  Used Avail Use% Mounted on
/dev/sda2        15G  6.4G  7.6G  45% /
```

```
[03-04-2026 09:53:39] {44fcaf38} student@vm-amd:~$ rm ~/overlay_31/merged/31_original.txt
[03-04-2026 09:53:46] {44fcaf38} student@vm-amd:~$ ls /home/student/overlay_31/merged
[03-04-2026 09:53:48] {44fcaf38} student@vm-amd:~$ ls -la /home/student/overlay_31/merged
total 8
drwxrwxr-x 1 student student 4096 Apr  3 09:53 .
drwxrwxr-x 6 student student 4096 Apr  3 09:39 ..
[03-04-2026 09:53:56] {44fcaf38} student@vm-amd:~$ echo "Восстановленный файл" > ~/overlay_31/merged/31_original.txt
[03-04-2026 09:54:02] {44fcaf38} student@vm-amd:~$ ls -la /home/student/overlay_31/merged
total 12
drwxrwxr-x 1 student student 4096 Apr  3 09:54 .
drwxrwxr-x 6 student student 4096 Apr  3 09:39 ..
-rw-rw-r-- 1 student student   40 Apr  3 09:54 31_original.txt
[03-04-2026 09:54:03] {44fcaf38} student@vm-amd:~$ vim overlay_check.sh
[03-04-2026 09:54:25] {44fcaf38} student@vm-amd:~$ chmod +x overlay_check.sh
[03-04-2026 09:55:05] {44fcaf38} student@vm-amd:~$ ./overlay_check.sh
Report saved to /home/student/overlay_31/31_audit.log
[03-04-2026 09:55:10] {44fcaf38} student@vm-amd:~$ cat /home/student/overlay_31/31_audit.log
OverlayFS audit report
======================
Whiteout files:

Differences between lower and merged:
diff -r /home/student/overlay_31/lower/31_original.txt /home/student/overlay_31/merged/31_original.txt
1c1
< Сригинальный текст из LOWER
---
> Восстановленный файл
```

```bash
#!/bin/bash

BASE=~/overlay_31
UPPER=$BASE/upper
LOWER=$BASE/lower
MERGED=$BASE/merged
LOG=~/overlay_31/31_audit.log

echo "OverlayFS audit report" > $LOG
echo "======================" >> $LOG

echo "Whiteout files:" >> $LOG
find $UPPER -name ".*" >> $LOG

echo "" >> $LOG
echo "Differences between lower and merged:" >> $LOG
diff -r $LOWER $MERGED >> $LOG

echo "Report saved to $LOG"
```

Ответы на вопросы

1. **Как OverlayFS скрывает файлы?** OverlayFS использует whiteout-файлы в upper слое. При удалении файла создаётся специальный скрытый файл, который «перекрывает» файл из lower слоя.
2. **Если удалить work?** Нет, перемонтировать нельзя. Каталог work обязателен для работы OverlayFS, так как используется для служебных операций.
3. **Если upper пуст?** Merged слой будет полностью соответствовать lower слою, так как изменений нет.

Часть 11. Оптимизируйте Dockerfile для приведенного ниже приложения app.py

- Использован образ python:3.12-slim.
- Dockerfile оптимизирован:
  - фиксирована версия
  - используется отдельный пользователь
  - кэширование зависимостей - добавлен .dockerignore - Сборка: docker build -t flask-app-31 .
- Запуск: docker run -d -p 5000:5000 -e STUDENT_NAME=Student31 flask-app-31
- Проверка: curl http://localhost:5000

Dockerfile

```
FROM python:3.12-slim
WORKDIR /app
RUN useradd -m -r appuser
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY app.py .
RUN chown -R appuser:appuser /app
USER appuser
EXPOSE 5000
CMD ["python", "app.py"]
```

requirements

```
flask==3.0.3
```

.dockerignore

```
__pycache__/
*.pyc
*.pyo
*.pyd
venv/
.env
.git/
.gitignore
Dockerfile
docker-compose.yml
```

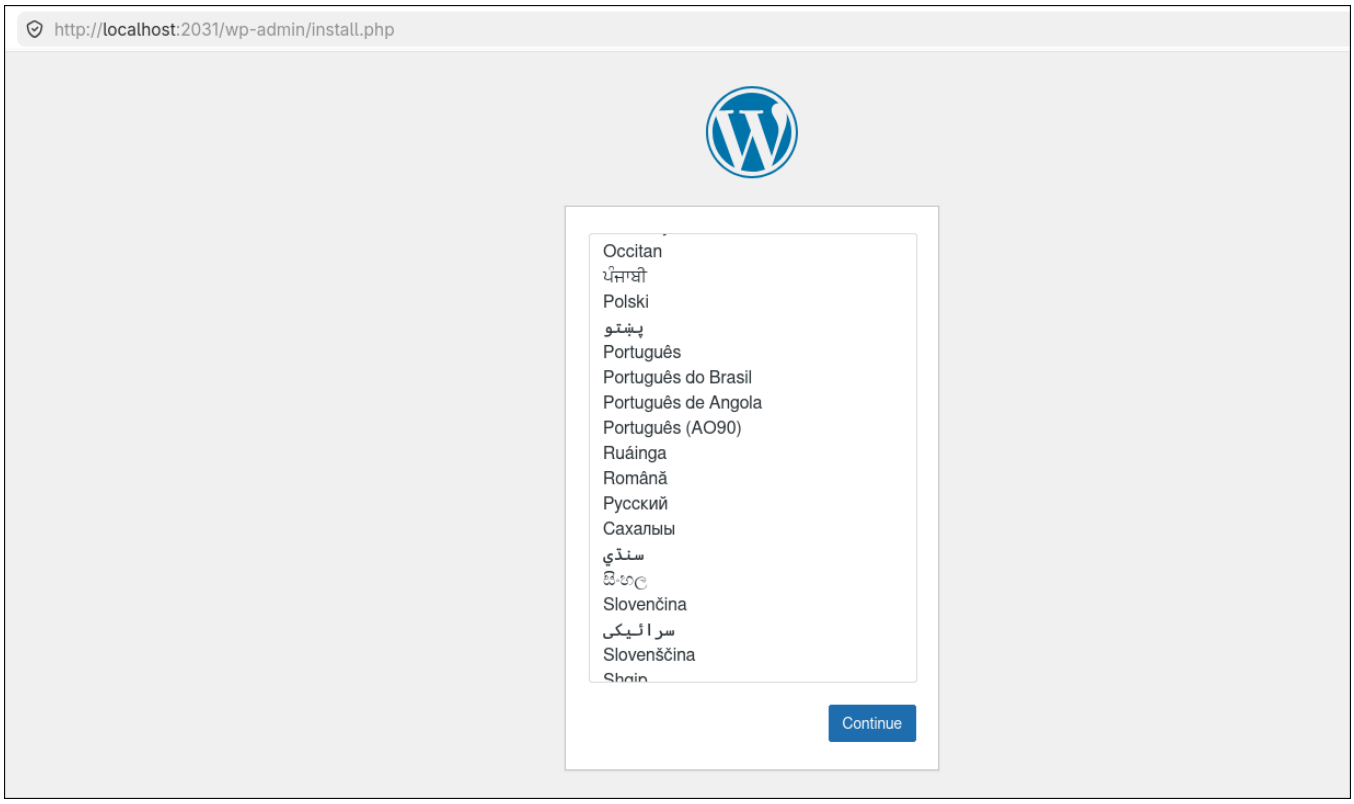
```
[03-04-2026 10:03:44] {44fcaf38} student@vm-amd:~/task11$ ls -la
total 24
drwxrwxr-x 2 student student 4096 Apr  3 10:02 .
drwxr-x--- 9 student student 4096 Apr  3 10:02 ..
-rw-rw-r-- 1 student student  384 Apr  3 09:59 app.py
-rw-rw-r-- 1 student student  243 Apr  3 10:02 Dockerfile
-rw-rw-r-- 1 student student   89 Apr  3 10:02 .dockerignore
-rw-rw-r-- 1 student student   13 Apr  3 10:02 requirements.txt
[03-04-2026 10:03:47] {44fcaf38} student@vm-amd:~/task11$ docker build -t flask-app-31 .
DEPRECATED: The legacy builder is deprecated and will be removed in a future release.
            Install the buildx component to build images with BuildKit:
            https://docs.docker.com/go/buildx/

Sending build context to Docker daemon   5.12kB
Step 1/10 : FROM python:3.12-slim
3.12-slim: Pulling from library/python
ec781dee3f47: Pulling fs layer
b1a20e2fae4c: Pulling fs layer
a6d1911b36ac: Pulling fs layer
e3c59d77c03e: Pulling fs layer
e3c59d77c03e: Waiting
b1a20e2fae4c: Verifying Checksum
b1a20e2fae4c: Download complete
a6d1911b36ac: Verifying Checksum
a6d1911b36ac: Download complete
e3c59d77c03e: Verifying Checksum
e3c59d77c03e: Download complete
ec781dee3f47: Verifying Checksum
ec781dee3f47: Download complete
ec781dee3f47: Pull complete
b1a20e2fae4c: Pull complete
a6d1911b36ac: Pull complete
e3c59d77c03e: Pull complete
Digest: sha256:3d5ed973e45820f5ba5e46bd065bd88b3a504ff0724d85980dcd05eab361fcf4
Status: Downloaded newer image for python:3.12-slim
 ---> fb1118f126b5
Step 2/10 : WORKDIR /app
 ---> Running in ffdcf8ef3e1b
 ---> Removed intermediate container ffdcf8ef3e1b
 ---> dd466ba9300e
Step 3/10 : RUN useradd -m -r appuser
 ---> Running in 9f61002c1bce
 ---> Removed intermediate container 9f61002c1bce
 ---> b6b6a50355cc
Step 4/10 : COPY requirements.txt .
 ---> a4099c28bd3f
Step 5/10 : RUN pip install --no-cache-dir -r requirements.txt
 ---> Running in dd5c6e2c45da
Collecting flask==3.0.3 (from -r requirements.txt (line 1))
```

http://localhost:2031

Container IP: 172.17.0.2 Student: Student31

Часть 12. Установка платформы публикации WordPress с помощью Docker Compose

- Создан docker-compose.yml.
- Использованы параметры:
  - порт: 2031
  - пароль БД: student_db_pass
  - том: student-wp-data
- Запуск: docker compose up -d
- Проверка: docker ps
- После запуска созданы сети:
  - docker0
  - br-xxxx
- Созданы volumes:
  - student-wp-data
  - db_data_31

```
[03-04-2026 10:18:47] {44fcaf38} student@vm-amd:~/task12$ vim docker-compose.yml
[03-04-2026 10:18:53] {44fcaf38} student@vm-amd:~/task12$ docker-compose up -d
Creating network "task12_default" with the default driver
Creating volume "task12_db_data_31" with default driver
Creating volume "task12_daniil-wp-data" with default driver
Creating wp_db_31 ... done
Creating wordpress_31 ... done
[03-04-2026 10:19:18] {44fcaf38} student@vm-amd:~/task12$ docker ps
CONTAINER ID   IMAGE                  COMMAND                  CREATED          STATUS                    PORTS                                      NAMES
0a6ac8cd3fe9   wordpress:php8.2-apache   "docker-entrypoint.s…"   11 seconds ago   Up 11 seconds             0.0.0.0:2031->80/tcp, [::]:2031->80/tcp   wordpress_31
240c646cb9c6   mariadb:11.4           "docker-entrypoint.s…"   32 seconds ago   Up 31 seconds (healthy)   3306/tcp                                   wp_db_31
[03-04-2026 10:19:29] {44fcaf38} student@vm-amd:~/task12$ docker volume ls
DRIVER    VOLUME NAME
local     task12_daniil-wp-data
local     task12_db_data_31
```